

FRED TALKS

30th May 2026

CONTENTS

Every app you've built is an ETL pipeline

LINELL BONNETTE · 2104 WORDS

Agentic search models

DOUG TURNBULL · 803 WORDS

The future of enterprise software

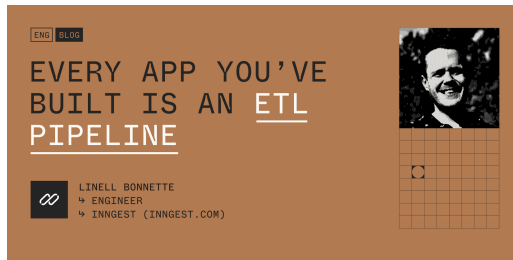
AARON LEVIE · 1851 WORDS

"Collaboration" is bullshit.

WESTENBERG. · 1380 WORDS

Every app you've built is an ETL pipeline

LINELL BONNETTE · 19 FEB 2026 · [SOURCE](#)



Featured image for Every app you've built is an ETL pipeline blog post

(you just didn't call it that)

ETL pipelines are everywhere. But they're also kind of a pain when handled incorrectly. What starts as a simple background job quietly absorbs status tracking, retry logic, and idempotency until you're maintaining a bespoke orchestration system instead of building your product. Step-level durability breaks this cycle. Here's how.

Every web app is an ETL system

I've spent my career building startups that all have one thing in common: they're ETL (Extract, Transform, Load) pipelines with a UI on top. Different domains, different customers, different use cases—all the same problem: *Pull data from somewhere, do something to it, put it somewhere else.*

If you've ever built a web application that did the same—you've built an ETL system. You may have called it an "integration" or a "sync job," but once you see the pattern, you can't unsee it. That admin CSV upload feature? Extract the file contents, transform the rows into something clean, load them into the database. The Stripe webhook handler? Same thing—receive the payload, map it to your billing state, update the record. It's all ETL.

And it doesn't matter how these systems are architected—they all beg the same questions:

- *What happens when the extract fails?*
- *How do you know the transform produced the right output?*

- *What if the load only partially succeeds?*
- *Can you retry without duplicating data?*
- *How do you even know the sync is running at all?*

And the one that inevitably comes up at the worst possible moment:

- *Why is the data like **this**?*

How LLM-powered features make ETL problems worse

With AI features showing up on every product roadmap, this pattern is about to get a lot more common. And honestly, you'd be surprised at how often AI-powered features are really just ETL wearing a mustache: If you extract a support ticket, ask an LLM to categorize it, and load that category back as structured data—that's ETL. Even RAG is just multiple extract-transform-load cycles chained together. Each one carries all the same failure modes. With one critical difference: the transform is non-deterministic. The same input can produce many different outputs, invalid schemas, or outright failures depending on the model's mood that millisecond.

And let's not forget the cost: Retries that re-run the whole pipeline burn through budget, without giving you any insight into what happened during that specific call. Was it a timeout? A rate limit? Did the model return something that doesn't match your schema? You can't debug that by grepping logs, so you fix in the dark, and re-run it all from scratch.

Multiply by thousands of records and all of a sudden you're choking your worker pool, exhausting your resources, and urgently needing to scale up cloud services.

The evolutionary trap: how three lines of code become an infrastructure project

Most ETL-in-disguise-projects follow *vaguely* the same trajectory. If you've built a few of these, I'm sure that it will feel familiar.

You've got your model and an API call that returns data you'll use to update it. Let's go with an easy one: `supportThread.fetchAllReplies()` reaches out and returns all of the comments on a customer support thread, and you want to use the result to categorize the type of ticket. So you add a background job:

```

async fetchThreadRepliesAndCategorize(supportThread) {
  const allReplies = await supportThread.fetchAllReplies()
  const category = await llmService.fetchCategoryForReplies(allReplies)
  await supportThreadRepo.update(supportThread, { category });
}

```

Three *clean* lines. It works so well that you get thousands of eagerly paying customers, and life is great!

Until it's not. One morning you wake up and realize that no threads have been categorized since yesterday. Was the job running? Did the provider go down? You've got no idea, because nothing tracks what happened. So you add a `categorization_status` column and wire up another background job that periodically retries failures.

```

async fetchThreadRepliesAndCategorize(supportThread) {
  try {
    await supportThreadRepo.update(supportThread, {
      categorization_status: "processing"
    })

    const allReplies = await supportThread.fetchAllReplies()
    const category = await llmService.fetchCategoryForReplies(allReplies)

    await supportThreadRepo.update(supportThread, {
      category,
      categorization_status: "complete",
      categorized_at: new Date()
    })
  } catch (error) {
    await supportThreadRepo.update(supportThread, {
      categorization_status: "failed",
      categorization_error: error.message
    })
  }
}

```

Not *as* clean, sure, but it's okay. Our simple three-line function is now sandwiched by bookkeeping for the categorization status, our support thread model is bloated with status columns that don't actually relate to the support thread's domain, and you now have an entirely new background job for retries but... it works. *Right?*

When retries create data duplication bugs

Let's continue that scenario. A month goes by. You're on an island in the Bahamas when your Director of Customer Support tells you they can't access the admin backend. You dig in and discover that a "failing" job has been retrying for two weeks straight, appending duplicate tags to the same thread on every pass. There are now millions of duplicated records in the database, everything is catastrophically slow, and *your margarita's ice **completely** melts while you fix that bug.*

So you come back from vacation and add idempotency checks, a deduplication pass, and a timeout.

```
async fetchThreadRepliesAndCategorize(supportThread) {
  // don't process if already in progress
  if (supportThread.categorization_status === "processing") {
    if (supportThread.updated_at > minutesAgo(30)) return
    // ...unless it's been stuck for 30 minutes, then retry
  }

  try {
    await supportThreadRepo.update(supportThread, {
      categorization_status: "processing"
    })

    const allReplies = await supportThread.fetchAllReplies()
    const category = await llmService.fetchCategoryForReplies(allReplies)

    // validate before saving – learned this the hard way
    if (!VALID_CATEGORIES.includes(category)) {
      throw new Error(`LLM returned invalid category: ${category}`)
    }

    // clear any duplicate tags from previous failed runs
    await supportThreadRepo.clearTags(supportThread)

    await supportThreadRepo.update(supportThread, {
      category,
      categorization_status: "complete",
      categorized_at: new Date(),
      categorization_error: null
    })
  } catch (error) {
    // handle error
  }
}
```

```

    })
  } catch (error) {
    await supportThreadRepo.update(supportThread, {
      categorization_status: "failed",
      categorization_error: error.message,
      categorization_attempts:
        supportThread.categorization_attempts + 1
    })

    // stop retrying after 5 attempts
    if (supportThread.categorization_attempts < 5) {
      retryQueue.enqueue(supportThread, { delay: "5m" })
    }
  }
}

```

Yea... remember when this was three lines?

Developers can't be the only ones to debug your pipeline

Here's the part that really stings: unless you build something to change that situation, *only a developer* has any insight into these problems. Your Director of Customer Support can't check whether categorization is healthy. Your ops team can't see which threads are stuck. You might have something like the Sidekiq dashboard that gives you *some* visibility into queue depths, but that tells you "a job failed," not "this specific support thread failed to categorize because the LLM returned an invalid category after three retries." You'll inevitably need to build an observability dashboard so non-technical users can understand how data flows through your system. And that dashboard is an entire project unto itself.

This is the evolutionary trap. Each fix is reasonable in isolation. Status tracking? Of course. Retry logic? Obviously. Idempotency? Learned that one the hard way. But each fix adds accidental complexity, each fix pulls you further from your actual product, and at the end of the cycle you're maintaining a bespoke orchestration system that nobody fully understands. You wanted to categorize support tickets. Now you're in the infrastructure business.

Building durable ETL pipelines with step-level retries

It's easy to assume that I joined Inngest entirely because being on the "execution team" sounded *really cool*, but the truth is that I joined because as soon as I saw the product I understood how valuable it would be for building things in the future. So yes, I *am* biased, but it *is* literally why I took the job. Here's that same support ticket code built with Inngest:

```
inngest.createFunction(  
  {  
    id: "categorize-support-thread",  
    singleton: { // only run this function once per thread_id  
      key: "event.data.thread_id",  
      mode: "skip" // skip any other matching events  
    },  
  },  
  { event: "support/thread.created" },  
  async ({ event, step }) => {  
  
    const replies = await step.run("fetch-replies", async () => {  
      return await fetchAllReplies(event.data.threadId)  
    })  
  
    const category = await step.run("classify-with-llm", async () => {  
      const result = await llmService.fetchCategoryForReplies(replies)  
      if (!VALID_CATEGORIES.includes(result)) {  
        throw new Error(`Invalid category: ${result}`)  
      }  
      return result  
    })  
  
    await step.run("save-classification", async () => {  
      await supportThreadRepo.update(event.data.threadId, { category  
    })  
  
    if (category === "urgent") {  
      await step.run("notify-team", async () => {  
        await notifySlack(event.data.threadId, category)  
      })  
    }  
  }  
})
```

```
}  
)
```

But look at what's *not here*.

Your support thread model goes back to representing a support thread, because the state of the *pipeline* is tracked by Inngest, not by your domain model.

No retry logic. If the LLM call fails, Inngest retries *that step*. It doesn't re-fetch the replies. It doesn't re-run the whole function from scratch. Each `step.run` is a checkpoint: if it succeeded, its result is saved, and the function picks up from where it left off. That margarita-ruining bug where failed retries duplicated data? Can't happen, because the `save-classification` step already completed and won't run again.

No hand-rolled idempotency. No stuck job detection. No retry counter. No deduplication pass to clean up after your own retry logic.

And the observability question, "how do I know what's happening without building a dashboard?", is answered before you even ask it. Every function run is visible in Inngest's dashboard: which event triggered it, which steps completed, which step failed, what the input and output were at each step, and how long each step took. Your Director of Customer Support doesn't need a developer to investigate. The state of every pipeline run is right there.

Event-driven architecture: how one event triggers multiple independent pipelines

There *is* one detail in the Inngest function above that's easy to gloss over but changes *everything* about how your system grows:

That function doesn't know *what* creates support threads. It doesn't care. It just knows that when a thread is created, it should categorize it. The thing that creates threads doesn't know this function exists either. They're completely decoupled, connected only by an event.

This matters because ETL systems don't stay simple. That categorization function is version one. Next month you also want to auto-generate a suggested response. The month after, you want to update a real-time analytics dashboard. Then you want to check whether the customer has an open billing dispute and escalate automatically.

In the evolutionary trap version, each of those additions means modifying existing code. Your `fetchThreadRepliesAndCategorize` function, already bloated with status tracking and retry logic, now needs to either do more things itself or know about more downstream jobs to enqueue. Every new feature increases the coupling and the blast radius of changes.

With Inngest, each of those is a new function that listens to the same event:

```
// These are entirely separate functions.
// They don't know about each other.
// Deploying one cannot break another.
```

```
inngest.createFunction(
  { id: "categorize-support-thread" },
  { event: "support/thread.created" },
  // ...classify and store
)
```

```
inngest.createFunction(
  { id: "generate-suggested-response" },
  { event: "support/thread.created" },
  // ...generate draft reply with LLM
)
```

```
inngest.createFunction(
  { id: "update-support-analytics" },
  { event: "support/thread.created" },
  // ...update dashboard metrics
)
```

```
inngest.createFunction(
  { id: "check-billing-escalation" },
  { event: "support/thread.created" },
  // ...check for open disputes, escalate if needed
)
```

That's fan-out: One event, multiple independent reactions. Adding new behavior means adding a new function and deploying it. You don't touch existing code. You don't risk breaking categorization by adding analytics. Each function has its own retry logic, its own observability, its own failure isolation. If the analytics function breaks, categorization keeps working.

This is the architecture that every bespoke ETL system eventually *wants* to be. The problem I kept hitting in my career wasn't that I didn't know event-driven was the right answer. It was that building the event infrastructure myself always turned into a massive undertaking that got compromised by deadlines and priorities. You end up with something half-event-driven and half-spaghetti, and the spaghetti always wins over time. Inngest gives you the clean version from day one without the infrastructure investment.

What changes when your ETL infrastructure is someone else's problem

What changed for me in practice is that all of the complexity I only cared about because I *had to* is gone. When you're building your support ticket categorization system, you can focus on the domain of support tickets instead of all of the *other stuff*. An entire category of hard problems becomes entirely boring, in the best possible way.

I think the biggest win with Inngest for ETL is *actually* more for product owners: the built-in observability and reporting that you get *for free* is something I've seen entire teams spend multiple quarters building themselves.

I've spent over a decade building ETL systems the hard way. Lots of hand-rolled moving parts supported by complex infrastructure and more than a couple of watered-down margaritas. Inngest is the good version of what I've been trying to build for my entire career, and I hope I get to see you build some super cool stuff with it.

Agentic search models

DOUG TURNBULL · 12 MAY 2026 · [SOURCE](#)

In retrieval, we know the lego pieces well. Embeddings. Rerankers. Query understanding. BM25. All the parts we put together in the standard stack.

But a new power is rising: **agentic search models**. LLMs trained *specifically* on controlling the search task. Compared to GPT-5 and pals, these models might focus on tasks / domains. They can be smaller, faster, and easier to deploy in our search infrastructure.

Let's set the stage.

With the traditional lego pieces, we build something like this thick search monolith:

image.png

That's what we've built for 1-2 decades. Queries flow into the system. We apply business rules and classify queries. We search one or more retrieval backends. We post-process and rerank what comes back.

It's all very manual, programmatic, and bespoke. Each piece doesn't see the whole. Every piece focuses on its part of the problem (rerankers, query classifiers, etc) ignorant of the whole.

Agentic search orchestrates the pieces

Agentic search unbundles the pieces to see the whole. An agent built on GPT-5 or Sonnet knows it has tools. It's given knowledge / context to solve the user's query.

The agent helps the user: driving a few simple retrieval primitives, exploring candidates to return what's relevant. Instead of thick, monolithic search, the underlying search tools become thin wrappers on our backend indices.

image.png

Unlike the traditional stack, **the agent sees the whole process**. It's not a series of reductive steps. Instead the model orchestrates a solution for the user using what's available - retrieval tools, other subagents, or a knowledge base to guide the user.

In other words, agentic search unbundles the retrieval stack. The parts still matter. But the whole can be managed by a single, intelligent model.

GPT-5 doesn't *really* know search

Frontier lab models (GPT-5, Sonnet and pals) do well in the 80% case. They understand the queries with general knowledge. They surface defensible search results.

But it's the last 20% that moves the needle.

What's in the last 20%? It's the non-obvious stuff in our users / domain. GPT-5 doesn't know that in our furniture store a search for "bistro tables" actually means "small outdoor tables" not restaurant equipment. GPT-5 doesn't know our fashion search users click on dark or plain patterns over complex ones.

Why not? Isn't GPT-5 itself also trained on search?

Yes, of course.

GPT-5 and pals think of 'search' as *web search*. Mainline models expect search tools that work near flawlessly. To give them answers they don't have. Unfortunately, our search isn't Google. We work on smaller, focused teams: building simpler search backends for our narrow domains. We need agents trained to reason and orchestrate simpler retrieval systems.

Currently, as I teach in [Cheat at Search with Agents](#), we build constraints + checks around the model. But this intense level of context engineering eats up tokens and costs \$\$.

Agentic search models for the last 20%

What if we could train an LLM to focus on search specifically? Even better: search in our domain, for our users?

We're beginning to see models in this direction: SID was first with [their SID-1 model](#). We've since seen Glean release [Waldo](#). Startups like Charcoal [tailor to your corpus](#).

Teams advertise these replacements by training on document *search* specifically - they focus on that last 20% needed to find useful results. Sid, for example, points to [smaller size and lower latency](#) compared to GPT-5 for agentic search.

Think about the implications. I've described a future where we don't build complex query + reranking pipeline. Instead agents orchestrate simpler retrieval primitives. A basic keyword search or embedding model with a few filters. Basic, scalable, and simple retrieval tools.

Then we deploy an agentic search model like SID-1. It knows what's relevant in our domain. It gets our users. Now it can orchestrate the tools to find relevant search results. Currently, sure, the focus begins with RAG and classic passage retrieval (ie chunks). That will surely expand to a family of models, targeting different domains from e-commerce to job search to ??.

Today we have scores of embedding models on huggingface, targeting financial, legal, e-commerce. Why wouldn't we also have many agentic search models? Tuned to domains?

An embedding only solves a narrow portion of the problem (similarity). An agentic search model could encompass the entire process from query understanding to hybrid search. Yes they're too slow today to drive site search. But that will surely change in the years ahead.

It's a future I'm excited about. Search will look radically different.

If you want to learn about these topics, check out my Cheat at Search w/ Agents class starting Monday.

Enjoy software.doug in training course form!

Starting May 18!

Signup here - <http://maven.com/software.doug/cheat-at-search> I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search, build better RAG and use LLMs in query understanding.

The future of enterprise software

AARON LEVIE · 28 JAN 2026 · [SOURCE](#)

One of the biggest topics right now in the tech world is what does the future of software look like when AI agents are doing a substantial amount of work in the enterprise. Does software get compressed and simply become a database that agents leverage? Do we use AI agents to code all our own personalized enterprise software in the future? Do startups or incumbents win this new battle? How big are software markets in a world of agents using these tools? And much much more.

It's impossible to try and tackle all of these questions sufficiently in one sitting, but I figured I'd lay out a bit of a roadmap for what enterprise software looks like in the future.

Why companies buy software

Firstly, we should all get on the same page about why companies use and buy enterprise software in the first place. You can think of enterprise software as a codification of a company's processes. These are the parts of your company that you *don't* want to change spontaneously, because they provide the structure for your workflows and operations to keep your business running, or allow you to do the other work that really matters.

A large car company decides to procure an ERP system because it's the backbone for complex global operations where failures could mean far more than the cost of the system. It's responsible for keeping track of tens or hundreds of billions in revenue, and the company wants to ensure that every single transaction that they make goes through the exact same way every

single time, with five 9s of uptime and reliability, and with an output that they can get accepted and approved by the auditors. Jobs are on the line, all the way up to the CEO, based on how well this system performs.

And the reason that companies choose to use software vendors that specialize in certain domains instead of just building this technology themselves can best be understood by the concept of “core” vs. “context,” popularized by Geoffrey Moore. The core stuff is what makes your company unique (like your core product, your talent, your culture, and so on), and the context is absolutely necessary, but largely non-differentiating between firms (think all of your systems of record like payroll, IT ticket responses, ERP systems, content management platforms, and so on).

For software that handles your company’s context activities, your customer will rarely notice when these things work great, but they’ll certainly notice the downstream impacts when they don’t. This is why you “rent” these services from software providers that do this as their day job for thousands or millions of other companies. And incidentally, much of this software helps define these underlying workflows in your organization so you don’t have to -- so each company doesn’t have to reinvent the wheel each on how to handle HR, IT ticketing, and so on.

Deterministic vs. non-deterministic systems

Ok, but if an enterprise is filled with 100X more AI Agents in the future, won’t that eventually consume all the use-cases that software did previously?

If you take the analogy of the industrial world, software is like the machinery in a manufacturing plant; and agents are like the people that operate that machinery. Just like machinery in a plant, in deterministic software, you want things to happen the same way, every time: you want data that goes into your ERP system to be calculated based on the business rules you’ve setup; you want security permissions you’ve set for accessing critical files to behave the same way every time, and not leak information between users; and you don’t want an HR system probabilistically generating new reporting structures for employees on every load.

Conversely, you want *non-deterministic* systems (in this case AI agents) for the things that people have always tended to do in these systems. An agent filling out notes from a user’s client meeting in a CRM system, generating a sales presentation for a new customer, answering a customer support inquiry with the best products to leverage, doing research or providing analysis on a set of enterprise documents and data for an important decision, and so on. As a result, the non-deterministic and deterministic elements of software end up working in a complementary fashion.

In contrast to some of the public discussion, I'd argue that in a world of 100X more AI agents than people in an enterprise, the value of the systems of record and tools agents will use will go up, not down. Because in this new world, software provides the guardrails on which agents can operate successfully within an enterprise, and gives them the underlying tools to use to be more productive themselves and work alongside people.

For instance, if you ask an AI agent to generate 10X the number of outbound email and marketing messages, where will all those agents find the data to work with, and where will the leads end up going to have a human sales rep act on them? Almost certainly some form of a CRM system. Or when an AI Agent processes reviews invoices and transactions, where will they enter this data when they're done to kick off a supply chain workflow? Again, an existing or reinvented ERP system. As long as we have humans and agents interacting with one another, there will need to be deterministic software that bridges these interactions seamlessly.

And as a result of all of these new agentic users within our software systems, the size of enterprise software markets are going to increase massively.

Unconstrained software markets

Traditionally, software companies have been stuck within the constraints of existing IT budgets, which tend to tap out around 3-7% of a company's revenue. This creates an inherent upper limit for what the budget can be for software, which translates into the total addressable market of various technology categories. Now, with AI Agents, the software is actually bringing along the work with the software, which means the budget software players are going after is the total spend that goes into doing that work in the company, not just the tech to enable it. This inevitably leads to a substantial increase in TAM for most software categories whose markets were artificially held back in size previously.

For instance, a contract management vendor just a few years ago was limited by the number of attorneys within an organization that needed to manage and review contracts. However, in a world of AI agents, that cap no longer exists, where you might have agents running continuously processing and reviewing contracts and providing extend legal services for any company. The size of the legal services market in the US is somewhere around \$400B, nearly 50-100X larger than the corresponding software categories in legal space. You can easily see how AI agents making the legal industry meaningfully more productive could capture multiple percentage points of that \$400B in spend, thus multiplying the size of the legal software market overnight.

The same analogy holds for almost any software category that was artificially constrained by a customer's headcount, or limited software spend, previously. We're already seeing this in spaces like coding, where the AI agents in coding are generating orders of magnitude more revenue for

the new coding startups than the prior IDE categories generated. And we can expect this same dynamic for most other areas of knowledge work over time, from healthcare and financial services to legal and marketing. Tools that bring along work will generate far more revenue than the prior era of tools used to.

The opportunity ahead for software

Now, the big open question is who wins in this technology wave. If we have learned anything from Clayton Christensen's Innovator's Dilemma framework, we know that incumbents tend to lose in markets when there's a new tech or business model innovation that is unattractive to them because it represents less revenue or profitability. Conversely, they tend to hold their own when the new business model or technology is supportive of their existing business, or perhaps even enhances it. Ultimately, in AI, we'll likely see examples of both outcomes.

Some existing software categories, like customer support or coding, are changing so rapidly that many incumbents will be unable to change their business models or technology stacks fast enough to respond to the growing demands of the new market, even if getting to the other side would be economically attractive. Similarly, as Jaya Gupta and Ashu Garg outlined in their piece on context graphs, there are many new categories of agent-native software that will need to be created for the first time because no existing vendor logically can capture the way the agent needs to do its work. This will create tremendous opportunities for new startups to build AI agents within these categories, and we're seeing new ones crop up every day.

Conversely, there are a number of existing SaaS players are in a natural position to power agents in their relevant work categories, especially those that have the natural context necessary for AI agents to be successful in automating work. Unsurprisingly, I'd expect there to be a correlation between the strength of a software company's moat and the amount of data they house, how complex the underlying workflows are, the number of connections there are with other systems, how intertwined a human-in-the-loop element is to the underlying agentic workflow, and how naturally AI agents can be plugged into these workflows (either as native users or via API in external agentic products).

Of course, this will mean that the business model of software (new upstarts and incumbents) will have to evolve over time. Today, the vast majority of SaaS products charge on a per-seat basis, which generally corresponds to most of the usage that the software sees today by its end users. But in a world where AI agents do most of the interaction and work on software, enterprise systems will have to evolve to support more of a consumption and usage-based model over time. AI agents don't cleanly fit as seats on software, because any given AI agent can do a varied amount of work within a system (e.g. you could have 1 agent doing a billion things or a billion agents doing one thing).

Inevitably, we'll see a mix of both outcomes. Users (or people) will continue to be seats within software, and AI agents that represent extended usage will be monetized through consumption. We're already seeing this play out in AI-native coding products and other platforms where there's an end-user seat component mixed with a consumption model on top. Obviously depending on the human vs. agent value proposition mix, the revenue stream will slide between these two ends of the continuum.

Ultimately, there's still plenty that needs to play out before we fully see what the future holds for enterprise software. But it will certainly be the most dynamic and exciting period we've ever seen in software history.

"Collaboration" is bullshit.

WESTENBERG. · 29 MAR 2026 · [SOURCE](#)



This newsletter is free to read, and it'll stay that way. But if you want more - extra posts each month, access to the community, and a direct line to ask me things - paid subscriptions are \$2.50/month. A lot of people have told me it's worth it.

Upgrade

In 1944, the Wehrmacht launched into Hitler's last ditch effort to save the Third Reich. The Battle of the Bulge was a doomed campaign and a doomed gamble from a doomed regime, but its brutality was a true second test of the US Army on the Western Front. During the battle, Army historian S.L.A Marshall began interviewing infantry companies who'd been baptised in combat. Published 3 years later in his 1947 book, Men Against Fire, Marshall's research showed that just 15-20% of riflemen in active combat positions ever fired their weapons - most kept their heads down. They moved when they were ordered and they held their positions, and they

mimicked the outward appearance of a soldier in battle - but shoot, they did not. By any standard organisational metric, the men were present and accounted for, but 4 out of 5 never pulled the trigger.

You can debate the extent of Marshall's numbers, and you can debate his methodology, but his ratio shows up, again and again. IBM stumbled onto it in the '60s when they discovered that 80% of computer usage came from 20% of the system's features. The pattern recurs because it describes something real about how effort is distributed inside groups, where a fraction of the people do most of the work, and the rest provide what you might ~charitably call "structural support."

Anyone who has worked in any large organisation knows exactly what I'm talking about.

The modern tech industry looked at the problem of human coordination and participation and decided the solution was "collaboration." If only 20% of us are operating with a "killer instinct" we need to be better at managing the shared instincts of the other 80%. And so collaboration became our shared obsession. We pursue "teamwork" as a holy grail.

The teamwork revolution, if you can call it that, gave us Notion for our documents, ClickUp for our tasks, Slack for our conversations, Jira for our tickets, Monday for our boards, Teams for the calls that should been emails, emails for the things that we couldn't squeeze in anywhere else, and now agents attempting to re-invent the whole stack. The average knowledge worker maintains accounts across system after system, switching between applications hundreds of times per day. And they produce, in aggregate, a staggering amount of coordinated and collaborative activity that never actually becomes anything resembling ~output.

When you strip away the product marketing and the dev relations and the blog posts and the funding rounds and the fuckery-upon-fuckery of it all, we're left with a simulation of collective engagement - but very little else. Transparency got confused with progress, visibility got confused with accountability, and being included in the thread became the same thing, socially and organizationally, as owning the outcome.

Once that confusion set in at the cultural level it became nearly impossible to dislodge. The feeling of collaboration is pleasant in a way that personal accountability can never be. Owning something means you, specifically and visibly you, can fail at it, specifically and visibly, in ways that attach to your name.

Collaborating means the failure belongs to the process.

So everyone chose collaboration, and we called it culture.

Marshall's riflemen were ordinary people responding to the diffusion of responsibility that happens inside any group. Maximilien Ringelmann measured the same phenomenon with ropes in 1913, long before there were Slack workspaces to offer an emoji-react to it. Individual effort drops predictably as group size increases. The presence of others dissolves the sense of personal responsibility in a way that feels, to everyone experiencing it, entirely reasonable. You're part of a team, you're contributing, you're also (measurably) pulling less hard than you would if the rope were yours alone. Every single person on the rope is doing this simultaneously, which is why the total force never adds up the way the headcount says it should.

Frederick Brooks identified the same dynamic in software development in 1975, watching IBM's System/360 project illustrate his emerging thesis that adding people to a late project makes it later. Communication overhead grows faster than headcount, coordination costs compound, and every new person contributes their capacity along with their relationships to everyone else. Those relationships require maintenance and produce misalignment and generate the need for more meetings to address the misalignment those meetings created.

Brooks might as well have described your company's Q3 roadmap planning cycle and your startup's sprint retrospective, all of which have gotten longer every year and produced, relative to their investment, less.

The collaboration industry has spent a fortune obscuring a dirty truth: most complex, high-quality work is done by individuals or very small groups operating with clear authority and sharp accountability, then rationalized into the language of teamwork afterward. Dostoevsky wrote *The Brothers Karamazov* alone. The Apollo Guidance Computer came from a team at MIT small enough to have real ownership, hierarchical enough that Margaret Hamilton's name could go on the error-detection routines she personally designed.

Communication matters, and shared context matters. But there's a huge difference between communication and collaboration as infrastructure to support individual, high-agency ownership, and communication and collaboration as the primary activity of an organisation. Which, if we're honest, is what most collaboration-first cultures have actually built. They've constructed extraordinarily sophisticated machinery for the social management of work, without actually doing the work they're socialising about.

If and when it exists, ownership looks like an individual who deeply gives a shit, making a call without waiting for group-consensus. That individual will be right sometimes, and they'll be wrong other times, and they'll own it. They won't sit around waiting to find out who has the authority to move a card from one column to another and post about it in the #celebrations channel.

But being that person sucks when “collaboration” is the reigning value, because every unilateral decision gets read as a cultural violation and a signal that you aren’t a team player.

Collaboration-as-ideology has made ownership and responsibility feel antisocial, which is a hell of a thing, given that ownership is the only mechanism that gets anything across the finish line.

You can see this excess everywhere. Standups where people announce their busy work and as long as everyone’s “on the same page” nobody changes course. Documents that are written to perform thinking so somebody else can perform thinking, with no decision in sight. Retros, and kickoffs, and WIP meetings that spawn their own retros, kickoffs and WIP meetings like cells dividing and re-dividing, with zero connection to the work that it’s nominally organising around.

Every project now seems to carry more coordination overhead than execution time, and when it fails the postmortem just recommends more collaboration...

At some point (and I think that point was fucking yesterday) we have to ask ourselves - what are we actually producing and who is actually responsible for producing it?

Because at some level, the answer for “who is responsible for X” has to be one single person, no matter how much the collaborative apparatus layered over modern work has been engineered to make that person invisible and dissolve accountability.

We need to find some path back to trusting that individuals will do their jobs, without every responsibility being visible to an entire organisation, without follow-ups being scheduled by a cadre of overpaid managers with their overfed metrics.

Maybe - just maybe - we could make our lives a little easier. Maybe we could let human beings keep their own lists of tasks, and we could let them sink or swim by how they manage those tasks, and we could assign blame to them and to them alone when they fuck up. Maybe we could do it without needing to have team-level views of every Kanban, calendar and task list. And maybe - if we let go of the warm, expensive fiction of collective endeavour - we could make it a little easier to see who among us are pulling the trigger and who are just keeping their heads down.